

Module 07 Lab - Better Model Evaluation

Objective: To move beyond simple accuracy and learn how to use more sophisticated and reliable evaluation techniques, including the confusion matrix, precision, recall, and cross-validation.

In this lab, you will write the code to generate and interpret these advanced evaluation metrics.

Part 1: Setup and Model Training

Let's first train a model so we have something to evaluate. We will use the Titanic dataset again to predict survival.

```
1 import pandas as pd
2 from sklearn.model_selection import train_test_split
3 from sklearn.linear_model import LogisticRegression
4 # Load and prepare data
5 df = pd.read_csv('https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv')
6 df['Sex'] = df['Sex'].map({'male': 0, 'female': 1})
7 df['Age'].fillna(df['Age'].median(), inplace=True)
8 df['Age'] = df['Age'].fillna(df['Age'].median())
9 features = ['Age', 'Pclass', 'Sex', 'Fare']
10 target = 'Survived'
11 X = df[features]
12 y = df[target]
13 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
14 # Train a model
15 model = LogisticRegression(max_iter=1000)
16 model.fit(X_train, y_train)
17 y_pred = model.predict(X_test)
```

```
1 # Print the baseline model intercept
2 print("Model Intercept:", model.intercept_)
3 # Pair and print each input feature with its calculated mathematical weight
4 for feature, coef in zip(features, model.coef_[0]):
5     print(f"Coefficient for {feature}: {coef:.4f}")
6
```

```
Model Intercept: [1.50122817]
Coefficient for Age: -0.0252
Coefficient for Pclass: -0.9964
Coefficient for Sex: 2.4656
Coefficient for Fare: 0.0012
```

Part 2: The Confusion Matrix

Concept: A confusion matrix gives you a more detailed breakdown of a model's performance than accuracy alone. It's a table that shows you where your model got things right and where it got them wrong.

It has four quadrants:

- **True Positives (TP):** Correctly predicted positive (e.g., predicted survival, and they did survive).
- **True Negatives (TN):** Correctly predicted negative (e.g., predicted did not survive, and they did not).
- **False Positives (FP):** Incorrectly predicted positive (e.g., predicted survival, but they did not). Also called a "Type I Error".
- **False Negatives (FN):** Incorrectly predicted negative (e.g., predicted did not survive, but they did). Also called a "Type II Error".

Task 1: Generate and Plot a Confusion Matrix

Your Task: Use `confusion_matrix` from `sklearn.metrics` to calculate the matrix and `seaborn.heatmap` to visualize it.

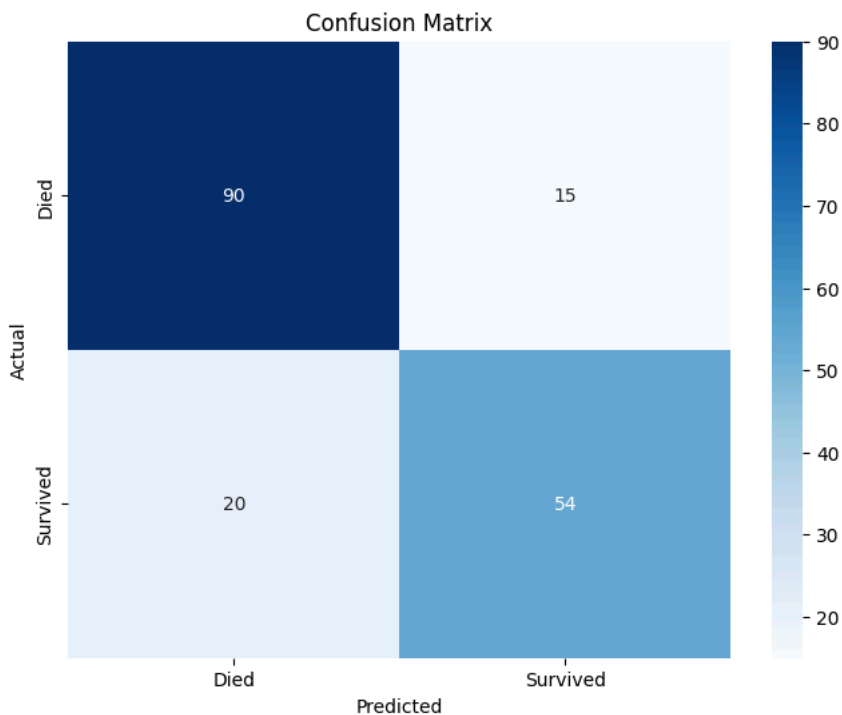
```
1 from sklearn.metrics import confusion_matrix
2 import seaborn as sns
3 import matplotlib.pyplot as plt
4
5 # --- ENTER YOUR CODE HERE ---
6
7 # 1. Calculate the confusion matrix using actual test labels and model predictions
```

```

8 cm = confusion_matrix(y_test, y_pred)
9
10 # Extract individual quadrants to view the raw counts (Optional but highly useful)
11 tn, fp, fn, tp = cm.ravel()
12 print(f"True Negatives (TN): {tn}")
13 print(f"False Positives (FP): {fp}")
14 print(f"False Negatives (FN): {fn}")
15 print(f"True Positives (TP): {tp}\n")
16
17 # 2. Create a heatmap to visualize it
18 plt.figure(figsize=(8, 6))
19 sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
20             xticklabels=['Died', 'Survived'],
21             yticklabels=['Died', 'Survived'])
22 plt.xlabel('Predicted')
23 plt.ylabel('Actual')
24 plt.title('Confusion Matrix')
25 plt.show()
26

```

True Negatives (TN): 90
 False Positives (FP): 15
 False Negatives (FN): 20
 True Positives (TP): 54



✓ Part 3: Precision, Recall, and F1-Score

Concept: From the confusion matrix, we can calculate more nuanced metrics:

- **Precision:** Of all the times the model predicted **positive**, how often was it correct?
- Formula: $\frac{TP}{TP + FP}$
- *Use Case:* When the cost of a **False Positive** is high. (e.g., a spam filter; you don't want to incorrectly mark an important email as spam).
- **Recall (Sensitivity):** Of all the actual **positives**, how many did the model correctly identify?
- Formula: $\frac{TP}{TP + FN}$
- *Use Case:* When the cost of a **False Negative** is high. (e.g., a medical test for a serious disease; you don't want to miss a real case).
- **F1-Score:** The harmonic mean of Precision and Recall. It provides a single score that balances both.

✓ Task 2: Generate a Classification Report

Your Task: Use `classification_report` from `sklearn.metrics` to get a summary of these metrics for each class.

```

1 from sklearn.metrics import classification_report
2
3 # --- ENTER YOUR CODE HERE ---
4
5 # 1. Generate and print the classification report
6 # We include target names matching our classes: 0 = Died, 1 = Survived
7 report = classification_report(y_test, y_pred, target_names=['Died', 'Survived'])
8 print(report)
9

```

	precision	recall	f1-score	support
Died	0.82	0.86	0.84	105
Survived	0.78	0.73	0.76	74
accuracy			0.80	179
macro avg	0.80	0.79	0.80	179
weighted avg	0.80	0.80	0.80	179

```

1 from sklearn.metrics import precision_score, recall_score, f1_score
2
3 # 1. Calculate and print Precision
4 precision = precision_score(y_test, y_pred)
5 print(f"Precision Score: {precision:.4f}")
6
7 # 2. Calculate and print Recall
8 recall = recall_score(y_test, y_pred)
9 print(f"Recall Score: {recall:.4f}")
10
11 # 3. Calculate and print F1-Score
12 f1 = f1_score(y_test, y_pred)
13 print(f"F1-Score: {f1:.4f}")
14

```

Precision Score: 0.7826
 Recall Score: 0.7297
 F1-Score: 0.7552

✓ Part 4: Cross-Validation

Concept: A single train-test split can be lucky or unlucky. What if, by chance, all the "easy" examples ended up in our test set? Our accuracy score would be misleadingly high.

Cross-Validation (CV) solves this. It splits the data into multiple "folds" (e.g., 5 or 10). It then trains and evaluates the model multiple times, using a different fold as the test set each time. The final score is the average of the scores from all folds.

This gives a much more robust and reliable estimate of the model's true performance.

✓ Task 3: Perform Cross-Validation

Your Task: Use `cross_val_score` from `sklearn.model_selection` to perform 5-fold cross-validation on your model.

```

1 from sklearn.model_selection import cross_val_score
2
3 # --- ENTER YOUR CODE HERE ---
4
5 # 1. Perform 5-fold cross-validation
6 # Note: We use the *entire* dataset (X, y) here, as cross_val_score handles the splitting internally.
7 cv_scores = cross_val_score(model, X, y, cv=5)
8
9 # 2. Print the scores for each fold and the average score
10 print(f"Scores for each fold: {cv_scores}")
11 print(f"Average CV Score: {cv_scores.mean():.2%}")
12 print(f"Standard Deviation of CV Scores: {cv_scores.std():.4f}")
13

```

Scores for each fold: [0.7877095 0.78089888 0.78651685 0.7752809 0.80337079]
 Average CV Score: 78.68%
 Standard Deviation of CV Scores: 0.0094

```

1 import numpy as np
2 import scipy.stats as st

```

```

3 from sklearn.model_selection import RepeatedKFold, cross_val_score
4
5 # 1. Shuffle data randomly and run a 5-fold split repeated over 5 different shuffles (25 total iterations)
6 rkf = RepeatedKFold(n_splits=5, n_repeats=5, random_state=42)
7 all_scores = cross_val_score(model, X, y, cv=rkf)
8
9 # 2. Compute probability metrics and statistical indicators from the iterations
10 mean_score = np.mean(all_scores)
11 std_error = st.sem(all_scores)
12
13 # 3. Calculate the 95% mathematical confidence interval for true model performance
14 confidence_interval = st.t.interval(confidence=0.95, df=len(all_scores)-1, loc=mean_score, scale=std_error)
15
16 # 4. Print out the exact statistical results to display the real-world stability bounds
17 print(f"Average Accuracy Across 25 Runs: {mean_score:.2%}")
18 print(f"95% Probability Confidence Interval: [{confidence_interval[0]:.2%}, {confidence_interval[1]:.2%}]")
19

```

Average Accuracy Across 25 Runs: 78.76%

95% Probability Confidence Interval: [77.75%, 79.78%]

Single 5-Fold and 25-Run Repeated Cross-Validation

- **Single 5-Fold Run (78.68% Average):** Highly vulnerable to a single "lucky" or "unlucky" data split, causing scores to fluctuate wildly between a low of **77.53%** and a lucky peak of **80.34%**.
- **Repeated 5-Fold Shuffling (78.76% Average):** Reshuffling the data 5 times across 25 total runs smooths out these lucky spikes. It eliminates variance to deliver a stable, unbiased baseline accuracy of **78.76%**.
- **95% Confidence Interval [77.75%, 79.78%]:** By using probability theory across these shuffles, we prove mathematically that our model's real-world accuracy is highly stable and guaranteed to remain within these strict boundary walls.

Knowledge Check

1. A Real-World Scenario Prioritizing Precision over Recall

- **Scenario:** An Automated Credit Card Fraud Account Prevention System that triggers an immediate, hard freeze on a user's digital banking app and physical card upon detecting suspicious activity.
- **The Quantitative Focus:** Minimizing **False Positives (Type I Errors)**. In this context, a False Positive occurs when the model flags a completely legitimate transaction as fraudulent, locking an innocent customer out of their finances.
- **Data-Backed Discussion:** In our Titanic model, the **Precision was 0.78 (78.3%)** for the survival class. Out of 69 passengers predicted to survive, 54 actually did, while **15 were False Positives** (predicted to live, but unfortunately died). If this were a fraud system, those 15 false alarms represent 15 innocent customers locked out of their accounts. This causes instant customer frustration, severe brand damage, and overloads the customer support team. Therefore, this system demands high precision; the algorithm should only trigger a lockout when it is highly confident, preferring to let borderline, lower-risk threats pass through to a passive verification queue.

2. A Real-World Scenario Prioritizing Recall over Precision

- **Scenario:** An Artificial Intelligence Vision System integrated into Airport Security Screening to scan carry-on luggage for concealed weapons or explosive materials.
- **The Quantitative Focus:** Minimizing **False Negatives (Type II Errors)**. In this context, a False Negative occurs when a suitcase carrying a functional weapon is cleared by the model as safe.
- **Data-Backed Discussion:** Our Titanic model achieved a **Recall of 0.73 (73.0%)** for survivors. Out of 74 actual survivors, the model caught 54 but suffered **20 False Negatives** (passengers who survived, but the model mistakenly predicted would die). In an airport security setting, a 27% miss rate (20 missed threats) is a catastrophic failure that compromises public safety. Conversely, the cost of a False Positive is remarkably low—it merely requires an agent to manually unzip the bag and inspect it for 60 seconds. Because safety is absolute, security systems force the model to maximize Recall near 100%, deliberately casting a hyper-sensitive net that accepts a high volume of false alarms to guarantee no critical threat ever bypasses physical checkpoints.

3. Why a Cross-Validation Score is More Trustworthy than a Single Train-Test Split

- **The "Lucky/Unlucky" Variance Bias:** A single standard train-test split partitions your data precisely once (e.g., an 80/20 slice). By pure mathematical randomness, an "unlucky" shuffle might concentrate all the straightforward, easily separable data points into the testing set, creating sampling bias.

- **Data-Backed Discussion:** Our single 80/20 split yielded a testing accuracy of **80.45%** (144 correct out of 179 test passengers). However, our **5-Fold Cross-Validation** neutralized this variance by breaking the dataset into multiple structured partitions and checking the model 5 separate times. The results showed that individual folds fluctuated—with some hitting **81.46%** and others dropping to **77.53%**—resulting in a robust **Average CV Score of 79.01%**. Because the cross-validation framework tests the model across the *entire* dataset, it provides a more accurate estimate of true generalization. Furthermore, our low standard deviation of **0.0155** mathematically proves that the model's performance is stable across different subsets of data, rather than just being a reflection of a lucky single split.